SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	N.Adi Narayana	**Reg. No.:**	192525346
Section / Batch:	AIML	**Date:**	08-08-2026

Code of Conduct

I G. Vignesh (192424153) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: Smart Mobile Keyboard Prediction System

A smartphone company is developing an AI-powered keyboard application that predicts the next word while users type emails, messages, and social media posts. The language model is trained on the corpus: *"Data science is powerful, data science drives innovation, data science is evolving."*
The development team employs Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve prediction accuracy and handle unseen word sequences. The following statistics are obtained from the corpus:

- C(data) = 3
- C(data science) = 3
- C(science is) = 2
- C(science drives) = 1
- C(science drives innovation) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word "science" are:

- P(is) = 0.66
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability P(science | data). Interpret how this probability influences next-word prediction in a mobile keyboard application.
2. A user types the unseen sequence "data science improves". Apply the Backoff Model and estimate the probability using lower-order n-grams. Explain why backoff is essential for real-time predictive text systems.
3. Using Deleted Interpolation, calculate the probability of the sequence "data science is". Discuss how interpolation balances reliability and coverage in sparse datasets.
4. Calculate the Entropy of the distribution P(is)=0.66 and P(drives)=0.33. Interpret the result and evaluate how entropy helps measure uncertainty and prediction confidence in intelligent keyboard systems.

Answers

Answer 1: MLE of P(science | data)

P(science | data) = C(data science) / C(data) = 3 / 3 = 1.0

The Maximum Likelihood Estimate of the bigram probability is 1.0 (100%). This means that in the training corpus, every single occurrence of the word "data" is immediately followed by "science", so the model has complete confidence in this transition. For the mobile keyboard predictor, this translates into the system instantly suggesting "science" the moment the user types "data", with a top-ranked, high-confidence suggestion chip. However, because the estimate is derived from only three occurrences, it is an example of data sparsity: a probability of exactly 1.0 is statistically fragile and may not generalise once the corpus grows to include

phrases such as "data entry" or "data analyst", which is precisely why smoothing and backoff techniques are needed alongside raw MLE.

Answer 2: Backoff Model for "data science improves"

The trigram "science improves" is unseen in the corpus, so C(science improves) = 0, and a plain MLE trigram probability would incorrectly be zero. The Backoff Model handles this by dropping to a lower-order n-gram whenever the higher-order count is missing:

Step 1 (Trigram): C(science improves) = 0 -> back off

Step 2 (Bigram): P(improves | science) = C(science improves) / C(science) = 0 / 3 = 0 -> back off

Step 3 (Unigram, with Laplace/add-one smoothing): P(improves) = (C(improves)+1) / (N+V) = (0+1) / (12+7) = 1/19 ~ 0.053

Here N = 12 total word tokens and V = 7 unique vocabulary words in the corpus. Because "improves" never occurs at all, even the unigram count is zero, so a discounting method (Laplace smoothing) is applied to avoid a hard zero probability. Backoff is essential for real-time predictive text because natural language is infinitely productive - users constantly type novel, unseen combinations, and a system that assigns zero probability to anything unseen would refuse to predict at all. Backing off to progressively simpler, more reliably estimated models keeps the keyboard responsive and lets it offer a plausible (if lower-confidence) suggestion instead of failing outright.

Answer 3: Deleted Interpolation for "data science is"

Deleted Interpolation combines trigram, bigram and unigram estimates using the given weights instead of choosing only one order:

P_interp(is | data, science) = λ1.P(is|data,science) + λ2.P(is|science) + λ3.P(is)

Trigram term: C(data science is) is not attested separately, so P(is|data,science) = 0

Bigram term: P(is|science) = C(science is)/C(science) = 2/3 = 0.667

Unigram term: P(is) = C(is)/N = 2/12 = 0.167

P_interp(is|data,science) = (0.5x0) + (0.3x0.667) + (0.2x0.167) = 0 + 0.200 + 0.033 = 0.233

P(data science is) = P(data) x P(science|data) x P_interp(is|data,science) = 0.25 x 1.0 x 0.233 = 0.058

Unlike backoff, which uses only one n-gram order at a time, deleted interpolation always blends all three orders. This balances reliability and coverage: the trigram model is the most context-specific but is unreliable when counts are sparse (as

here, where no trigram count exists), while the unigram model is always available but ignores context. By mixing them with fixed weights, the model still gives weight to the well-estimated bigram signal (science -> is) while never fully discarding the broader unigram statistics, producing a smoothed, non-zero probability that is more robust on small or sparse datasets such as this training corpus.

Answer 4: Entropy of P(is)=0.66, P(drives)=0.33

$H = -\Sigma\ p(x).\log_2 p(x) = -[0.66 \times \log_2(0.66) + 0.33 \times \log_2(0.33)]$

$\log_2(0.66) = -0.599,\ \log_2(0.33) = -1.600$

$H = -[0.66 \times (-0.599) + 0.33 \times (-1.600)] = -[-0.395 - 0.528] = 0.923$ bits

The entropy of this two-outcome distribution is approximately 0.923 bits, close to the maximum possible entropy of 1 bit for a binary choice (which occurs at a perfectly uniform 50/50 split). This indicates that the model is only mildly more confident about "is" than "drives" after the word "science" - the prediction carries real but limited certainty. In an intelligent keyboard, entropy is a direct, quantitative measure of prediction confidence: low entropy (close to 0) means the model is very sure of the next word and can auto-commit a single suggestion, whereas high entropy (close to 1 bit here) means the two candidates are nearly equally likely, so the UI should present both "is" and "drives" as separate suggestion chips rather than silently completing the word for the user.

CASE STUDY 2: AI-Powered Customer Support Chatbot

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses. Consider the following user inputs:

1. "Book a flight ticket now."
2. "This book is interesting."

The chatbot uses the Penn Treebank POS Tagset and a Hidden Markov Model (HMM) for probabilistic tagging.
Given:

- P(book | VB) = 0.6
- P(book | NN) = 0.4
- P(Start → VB) = 0.5
- P(Start → NN) = 0.5

The system must determine whether the word "book" functions as a Verb (VB) or Noun (NN) depending on context.

Questions

1. Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the tag assigned to the word "book" using contextual clues.
2. Using the HMM probabilities, compute the likelihood of "book" being tagged as a Verb (VB) in the sentence "Book a flight ticket now." Explain why the probabilistic model favors this tag.
3. Compare Rule-Based Tagging and Stochastic (HMM) Tagging in resolving lexical ambiguity. Recommend which approach is more suitable for large-scale chatbot deployment and justify your answer.
4. Evaluate the role of standardized POS tagsets and word classes in improving intent detection, response generation, and overall chatbot performance.

Answers

Answer 1: POS Tagging of Both Sentences

Sentence 1: Book/VB a/DT flight/NN ticket/NN now/RB ./.

Sentence 2: This/DT book/NN is/VBZ interesting/JJ ./.

In Sentence 1, "Book" appears as the first word of an imperative sentence, directly followed by the determiner "a" and the noun phrase "flight ticket" - this is the classic

command structure ("[You] Book a flight ticket now"), so "book" functions as a base-form Verb (VB) meaning "to reserve". In Sentence 2, "book" is preceded by the determiner "This" and followed by the verb "is", placing it in the subject noun-phrase position ("This book" = subject); here it is tagged as a singular common Noun (NN) meaning a physical/reading book. The surrounding context - specifically the preceding determiner versus the sentence-initial imperative position - is what resolves the lexical ambiguity of "book".

Answer 2: HMM Likelihood of "book" as VB

Score(VB) = P(Start→VB) x P(book|VB) = 0.5 x 0.6 = 0.30

Score(NN) = P(Start→NN) x P(book|NN) = 0.5 x 0.4 = 0.20

Comparing the two joint scores, Score(VB)=0.30 exceeds Score(NN)=0.20, so the Viterbi/HMM decision rule (argmax over tag sequences) selects VB as the most probable tag for "book" in "Book a flight ticket now." The probabilistic model favours VB because it multiplies the transition probability of starting a sentence with a verb with the emission probability of "book" given that tag, and both of those component probabilities are higher for VB than for NN in this context (0.6 emission for VB vs 0.4 for NN, with equal transition priors). This mirrors the linguistic intuition that airline-booking commands typically begin with an imperative verb.

Answer 3: Rule-Based vs Stochastic (HMM) Tagging

Rule-Based Tagging relies on hand-crafted linguistic rules (for example, "tag a word as a verb if it starts a sentence and is followed by a determiner") applied deterministically. It is transparent and easy to debug, but it does not scale well: writing rules to cover every ambiguous word and every context in a large, evolving chatbot vocabulary is labour-intensive and brittle to new phrasing. Stochastic (HMM) Tagging instead learns emission and transition probabilities from labelled corpora and picks the tag sequence that maximises overall sentence likelihood; it generalises automatically to sentence structures not explicitly anticipated by a rule-writer and adapts as more conversational data is collected. For a large-scale, multilingual, constantly-growing chatbot such as an airline's support system, HMM (or its modern neural successors) is the more suitable approach because it scales with data, captures probabilistic context rather than fixed patterns, and requires far less manual maintenance than an ever-expanding rule set - though a small layer of rule-based post-processing is often kept for handling clear-cut edge cases.

Answer 4: Role of Standardised POS Tagsets and Word Classes

A standardised scheme such as the Penn Treebank tagset gives every component of the chatbot pipeline a common, unambiguous vocabulary for grammatical roles. For intent detection, knowing that "book" is tagged VB (not NN) immediately signals an action/request intent ("make a reservation") rather than a topic about a

physical object, letting the intent classifier route the query correctly. For response generation, POS information helps the natural-language generation module produce grammatically correct, well-formed replies (matching verb tense, subject-verb agreement, and phrase structure). More broadly, consistent word-class identification underlies syntactic parsing, named-entity recognition, and slot-filling (e.g., separating the flight destination noun phrase from the action verb), all of which directly raise the accuracy, consistency, and naturalness of the chatbot's understanding and responses across the millions of varied user utterances it must handle.

CASE STUDY 3: News Analytics and POS Tag Correction System

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase.

The sentence processed by the system is:

"Economic growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and uncertainty evaluation.

Questions

1. Apply the transformation rule and update the POS tag sequence. Explain why the correction is linguistically appropriate.
2. Analyze the initial tagging errors and justify the corrected tag assignments using grammatical structure and sentence semantics.
3. Assuming the corpus contains the words:
    - economic (120 occurrences)
    - growth (450 occurrences)
    - increases (210 occurrences)
    - employment (380 occurrences)

    Compute the word frequency distribution and explain how frequency information supports probabilistic POS tagging.

4. Evaluate how Transformation-Based Tagging improves tagging accuracy compared with the initial tagging stage. Discuss how entropy can be used to measure uncertainty in tag assignments before and after rule application.

Answers

Answer 1: Applying the Transformation Rule

Initial: economic/JJ growth/NN increases/NNS employment/NN

Rule: Change NNS -> VBZ if the preceding word is tagged NN

Check: "increases" (NNS) is preceded by "growth" (NN) -> condition satisfied -> retag

Updated: economic/JJ growth/NN increases/VBZ employment/NN

The correction is linguistically appropriate because a singular common noun ("growth") acting as the grammatical subject cannot be directly followed by another plural noun (NNS) within a well-formed simple sentence - what follows a subject noun must be the sentence's main verb. "Increases" here is being used as a present-tense, third-person-singular verb ("growth increases employment"), which is exactly the VBZ category, so the transformation rule correctly repairs a tagging error introduced by the initial (context-blind) tagger.

Answer 2: Analysis of the Initial Tagging Errors

The initial tagger mis-labelled "increases" as NNS (plural common noun) almost certainly because "increases" superficially resembles a plural noun form (ending in "-s", like "prices" or "changes") - a common source of error for tag-frequency-based or lexicon-lookup taggers that do not use full sentence context. Grammatically, however, the sentence "Economic growth increases employment" has the structure [Adjective + Subject-Noun] + [Verb] + [Object-Noun]: "growth" is the subject, "employment" is the object, and there must be a finite verb linking them - a role that only "increases" can fill. Semantically too, the sentence describes growth causing a rise in employment, an action, which only a verb reading of "increases" (VBZ) captures correctly; the NNS reading would leave the sentence without any predicate at all, which is ungrammatical. The corrected VBZ tag is therefore justified both syntactically (subject-verb-object structure) and semantically (an action/change being described).

Answer 3: Word Frequency Distribution

Total corpus count = 120 + 450 + 210 + 380 = 1160

P(economic) = 120/1160 = 0.1034 (10.34%)

P(growth) = 450/1160 = 0.3879 (38.79%)

P(increases) = 210/1160 = 0.1810 (18.10%)

P(employment) = 380/1160 = 0.3276 (32.76%)

These relative frequencies are exactly the raw material for Maximum Likelihood-based probabilistic POS tagging: a stochastic tagger estimates emission probabilities such as P(word|tag) directly from how often a word occurs (and how often it occurs with each tag) in a large training corpus. Words like "growth" that occur very frequently (38.79% of this mini-sample) give the tagger a statistically reliable, low-variance estimate of their most likely tag, while rarer words such as "economic" (10.34%) yield noisier estimates and benefit more from smoothing. In this way, corpus-derived frequency statistics directly determine how much the

tagger "trusts" a given emission or transition probability when disambiguating tags across millions of news articles.

Answer 4: Transformation-Based Tagging vs Initial Tagging, and Entropy of Confidence

The initial (baseline) tagging stage typically assigns the single most frequent tag for each word in isolation, ignoring surrounding context; this is fast but error-prone in exactly the way seen with "increases" above. Transformation-Based Tagging (the Brill Tagger) starts from that same baseline but then applies an ordered set of learned, context-sensitive correction rules (like "NNS -> VBZ after NN"), iteratively fixing systematic errors. Because each rule is learned from data to reduce overall tagging error, accuracy improves substantially after each pass without requiring a full probabilistic model at run time, and the resulting rules remain human-readable and easy to audit - a practical advantage for a news-analytics pipeline processing millions of articles.

Entropy quantifies this improvement in confidence. Before any rule is applied, "increases" is genuinely ambiguous between NNS and VBZ; treating the two candidate tags as roughly equally likely gives an entropy close to the maximum for a binary choice:

H_before = -(0.5 log2 0.5 + 0.5 log2 0.5) = 1 bit (maximum uncertainty)

After the transformation rule deterministically resolves the tag to VBZ with certainty, the distribution collapses to a single outcome:

H_after = -(1.0 log2 1.0) = 0 bits (no uncertainty)

This drop from 1 bit to 0 bits is a direct, measurable illustration of how Transformation-Based Tagging increases tagging confidence: each successfully applied rule removes ambiguity from the tag distribution, so tracking entropy before and after rule application gives news-analytics engineers a quantitative way to monitor and validate the accuracy gains delivered by the Brill Tagger across the corpus.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____